

Custom GUI for DE

Author: Santhosh SL

Contact: san2909s@gmail.com | +91 95358 70426

GitHub: https://github.com/Santo29SL/CustomDE_UI_POC

Overview

This document provides a technical guide and architectural blueprint for the Custom Data Engineering Platform. The solution wraps **Mage OSS**, **Apache Superset**, and **PostgreSQL** in a unified Angular/.NET dashboard.

Index

1. Requirements
 2. Tech stack
 3. High-level design
 4. End-to-end flow
 - Step 1: Project initialization & setup (Path A / Path B)
 - Step 2: Settings configuration
 - Step 3: Data ingestion & Bronze script generation
 - Step 4: Coding & local script execution (Notebook editor)
 - Step 5: Orchestrating the Medallion run (Pipeline lineage DAG)
 - Step 6: Monitoring & verification (Superset embed)
 5. References
-

1) Requirements

1. Angular frontend + .NET backend.
2. Single portal that embeds: ingestion, notebook editor, file sidebar (project structure), pipeline builds, database tables + preview, inbuilt terminal, and visualization.

3. Orchestration engine: **Mage OSS** (Apache License 2.0).

Figure 1 — Mage OSS licence applicability (refer official links in reference section)

3. Mage OSS Enterprise Licensing & Compliance

The orchestration engine runs **Mage OSS**, which is released under the **Apache License 2.0**. This license is highly permissive and well-suited for enterprise-level deployment, subject to key compliance guidelines.

Apache License 2.0 Permitted Uses

- **Commercial Use:** You are free to use Mage OSS commercially, run internal data operations, and build proprietary wrappers around it.
- **Modification & Distribution:** You may modify the source code of Mage OSS and distribute it.
- **Patent Rights:** The Apache License includes an explicit grant of patent rights from contributors to users.

Enterprise Compliance Requirements

IMPORTANT
If you distribute Mage OSS as part of an enterprise application, you must adhere to these requirements:

1. **Notice & Copyright:** You must include a copy of the original license text and keep original copyright/patent notices intact in modified versions.
2. **State Changes:** Any modified file must carry prominent notices stating that you changed the code.
3. **NOTICE File:** If the original distribution contains a `NOTICE` text file, you must include it in your distribution.

Enterprise Liability Safeguards

WARNING

- **No Warranty:** Mage OSS is provided "AS IS", without warranties of any kind. Your enterprise assumes all risks related to performance and data security.
- **Liability Limitation:** In no event will contributors be liable for damages (including data loss or business interruption) arising from the use of the software.

2) Tech stack

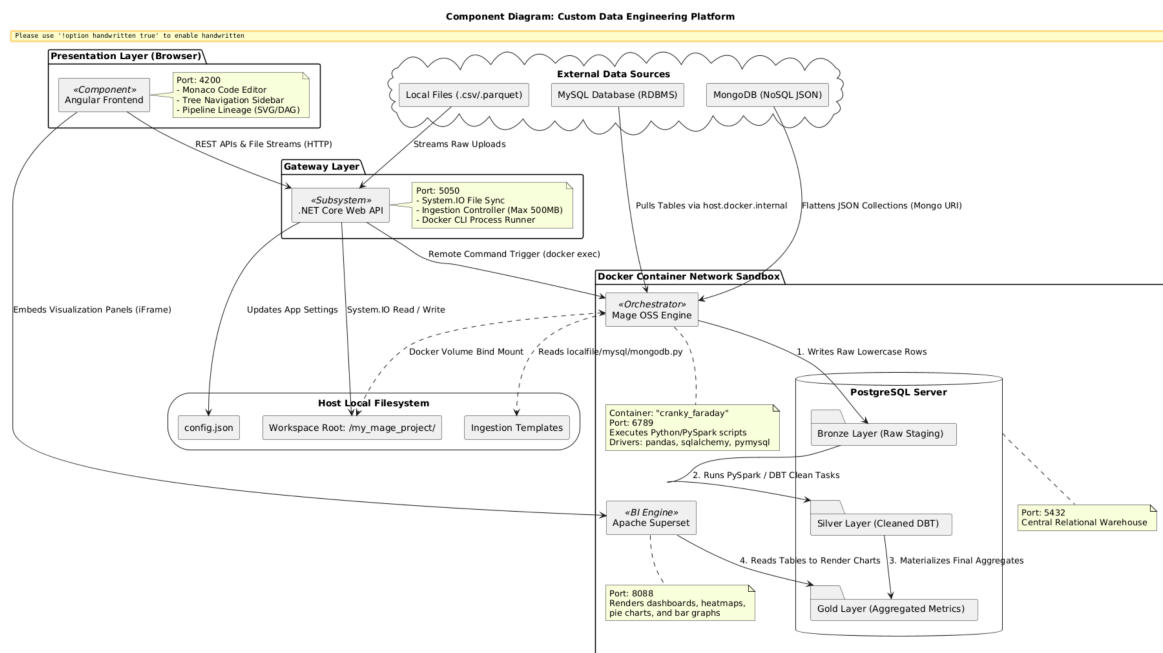
Figure 2 — Technology stack

Layer	Technology	Version / Specifics	Description
Frontend UI	Angular	18+ (TypeScript, CSS3, HTML5)	Premium UI, Monaco Editor, dynamic tree-nav, block-wise SVG animation layout.
Backend Gateway	.NET Core Web API	10.0 (C#)	Dynamic database manager, filesystem sync, and Docker orchestration wrapper.
Orchestration Core	Mage OSS	Latest Community	Apache 2.0 compliant container orchestrator.
Database	PostgreSQL	15+	Relational data warehouse utilizing Medallion isolation schemas.
BI & Analytics	Apache Superset	Community Edition	Rich visualization layer embedded directly inside the portal.
Container Engine	Docker	macOS / Desktop Engine	Isolates Mage, MongoDB, MySQL, and Superset services.

3) High-level design

1. **Mage OSS (Orchestration Engine):** Runs in the `cranky_faraday` Docker container. Preloaded with drivers/libraries such as `PyMySQL`, `PyMongo`, `cryptography`, `pandas`, `SQLAlchemy`, and Spark dependencies to orchestrate Python- and Spark-based ETL scripts.
2. **Angular frontend UI (Control center):** SPA hosting Monaco editor (IDE), terminal streaming stdout from container runtime, database preview grids, Medallion file explorer, dynamically rendered SVG DAG lineage graphs, and embedded dashboards.
3. **PostgreSQL server (Unified warehouse):** Project-level isolation. Gateway provisions one database per project with `bronze` (raw), `silver` (cleaned/conformed), and `gold` (business aggregations) schemas.
4. **Automated pipeline sync:** Mage scans `bronze/`, `silver/`, `gold/`, parses blocks, and auto-generates DAG dependencies.
5. **Apache Superset (Visualization embed):** Embedded via secure iframe, enabling charting and dashboarding on Gold tables.

Figure 3 — Architecture overview



4) End-to-end flow

Step 1 — Project initialization & setup flows

When you access the platform, you are greeted with the **Onboarding Wizard** if no project is active. You can also click **+ New Project Schema** in the header.

Figure 4 — Onboarding wizard

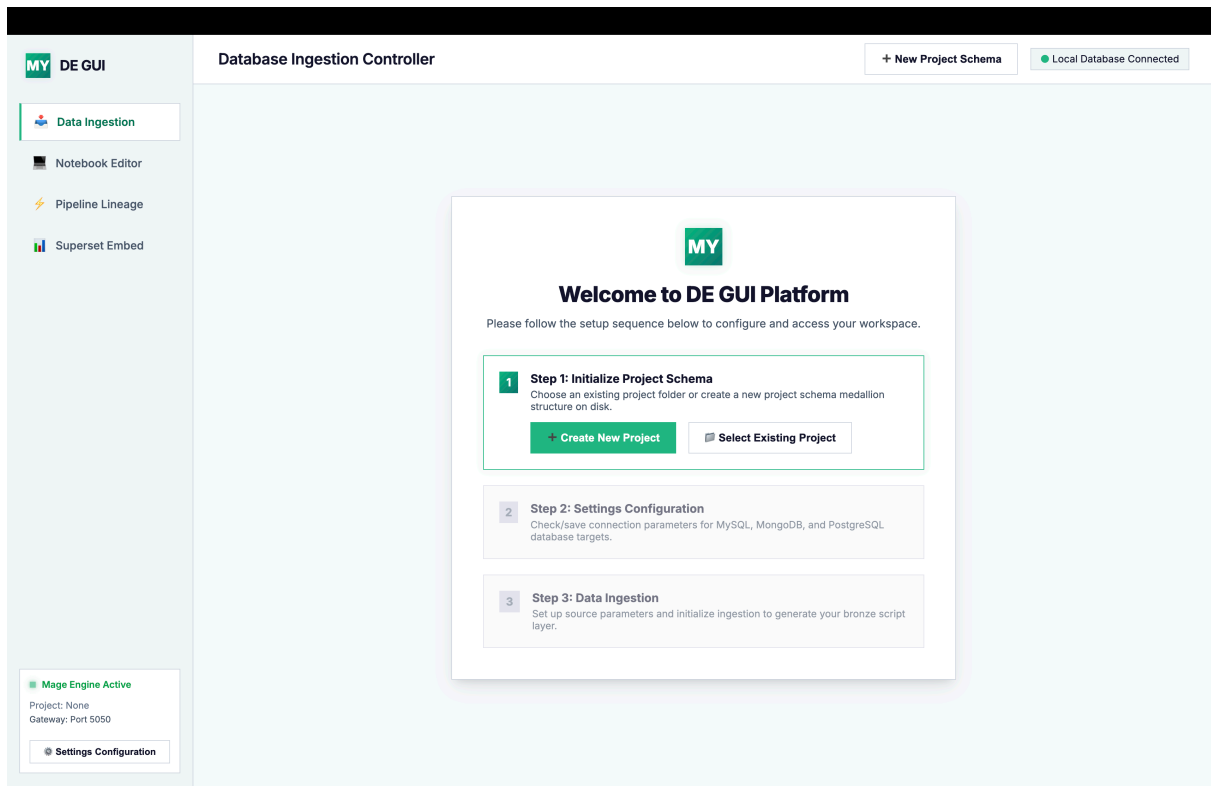
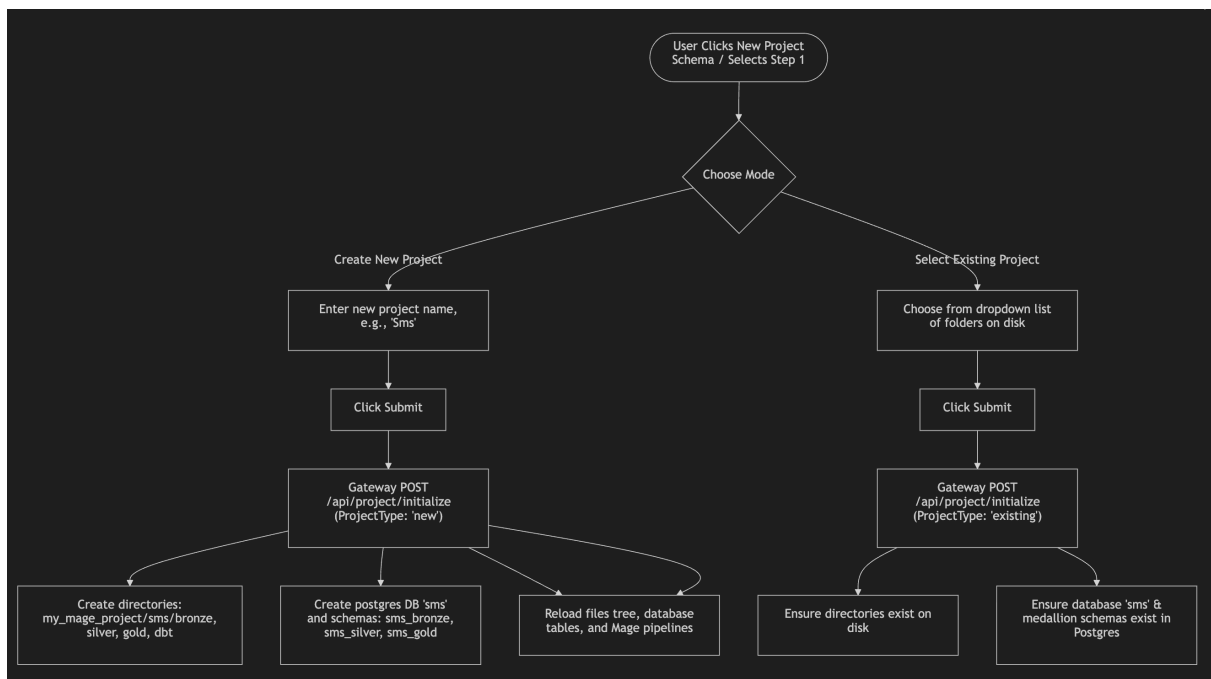


Figure 5 — Project schema dialog



Path A: Creating a new project schema

1. Click **+ New Project Schema** (header) or **Create New Project** (wizard).
2. Select **New Project**.

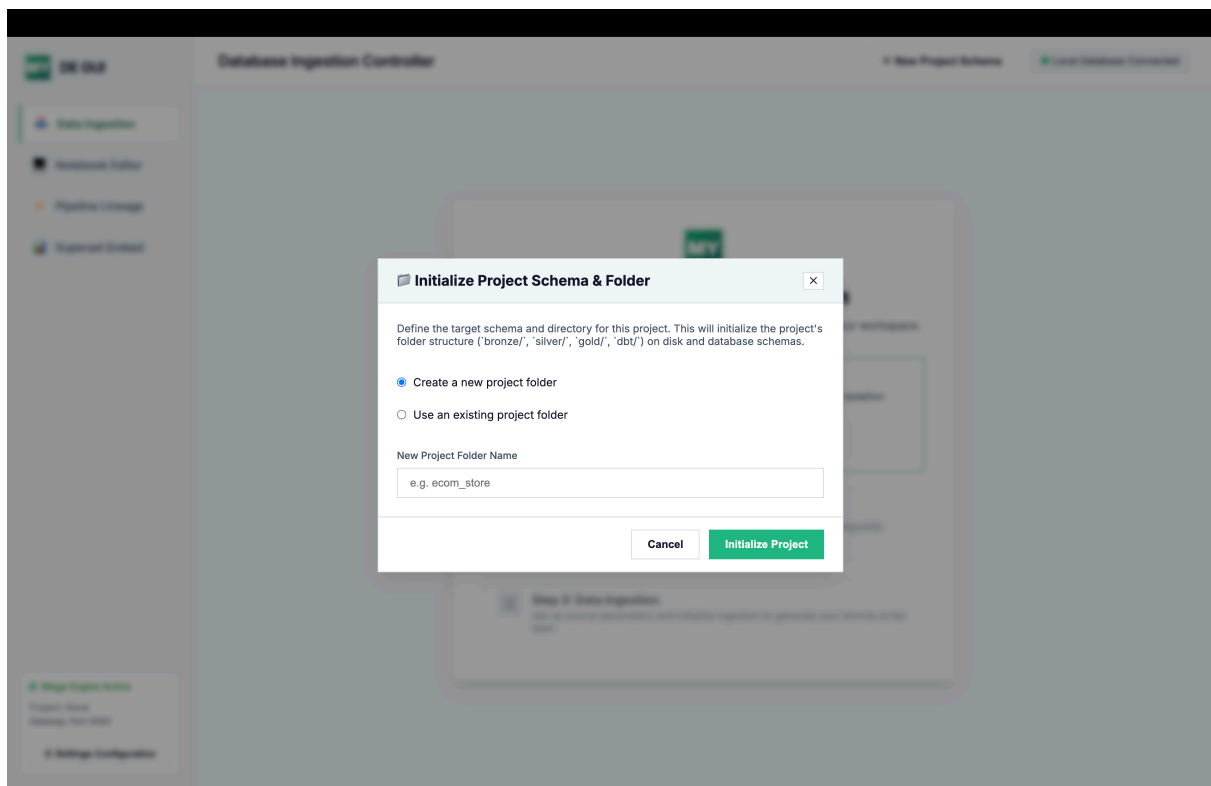
3. Enter a unique name (e.g., `Sms`).

4. Click `Initialize Project`.

Backend actions

- `POST /api/project/initialize` with `{ ProjectName: "Sms", ProjectType: "new" }`.
- Name sanitized to lowercase underscores (`sms`).
- Folders created: `my_mage_project/sms/bronze/`, `silver/`, `gold/`, `dbt/`.
- Database provisioned (if needed), with schemas created:
 - `sms_bronze`
 - `sms_silver`
 - `sms_gold`
- Gateway config updated to make `Sms` the active project.

Figure 6 — Path A: New project scheme select



Path B: Selecting an existing project schema

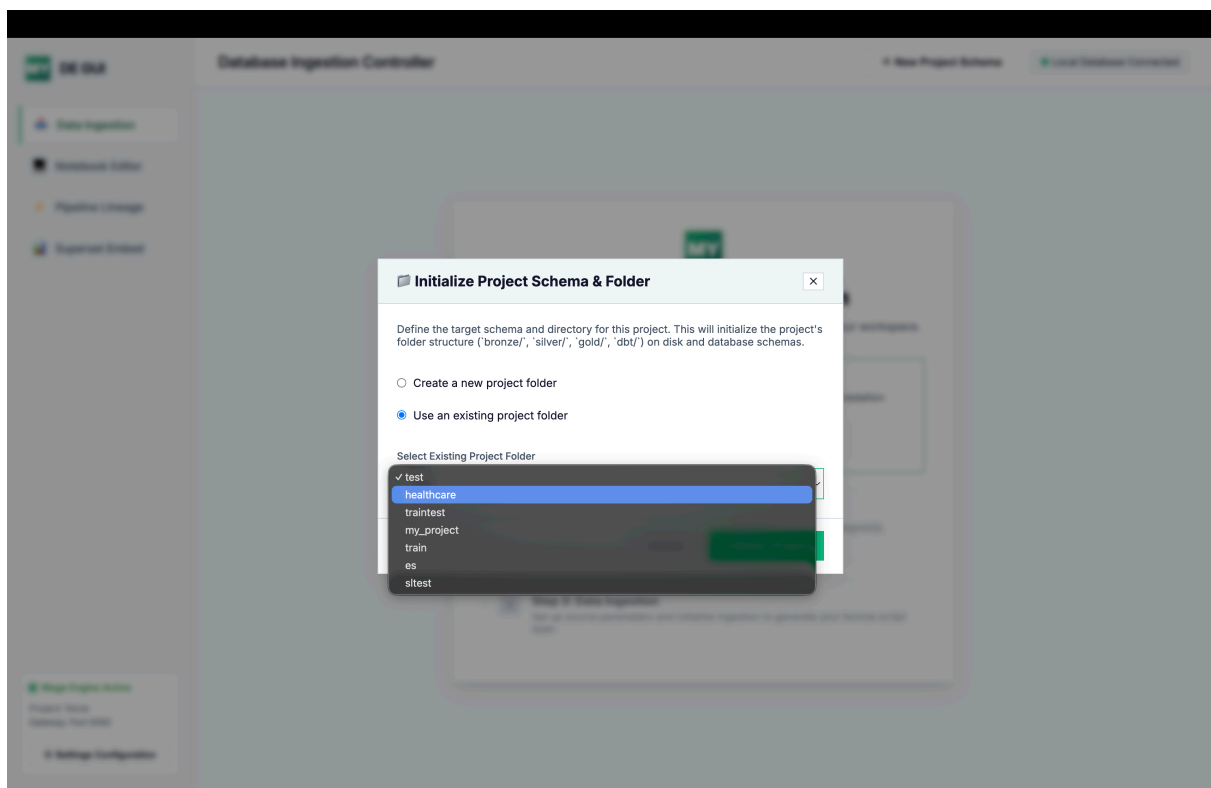
1. Click `Select Existing Project`.

2. Choose a project directory (scanned from `my_mage_project/`).
3. Click **Initialize Project**.

Backend actions

- `POST /api/project/initialize` with `{ ProjectName: "healthcare", ProjectType: "existing" }`.
- Re-provisions schemas and database structures if missing.
- Scans and loads existing scripts into the file explorer tree.

Figure 7 — Path B: Select existing project



Step 2 — Settings configuration

Before writing pipeline blocks, configure and save connection credentials.

1. Click **Settings Configuration**.
2. Choose execution mode: Docker container / local machine.
3. Provide connection details:
 - MongoDB connection URI

- MySQL server details (host, port, username, password, database)
- PostgreSQL destination URI
- Mage OSS URL

4. Click **Test Connections**, then **Save Configuration**.

Figure 8 — Local file ingestion settings

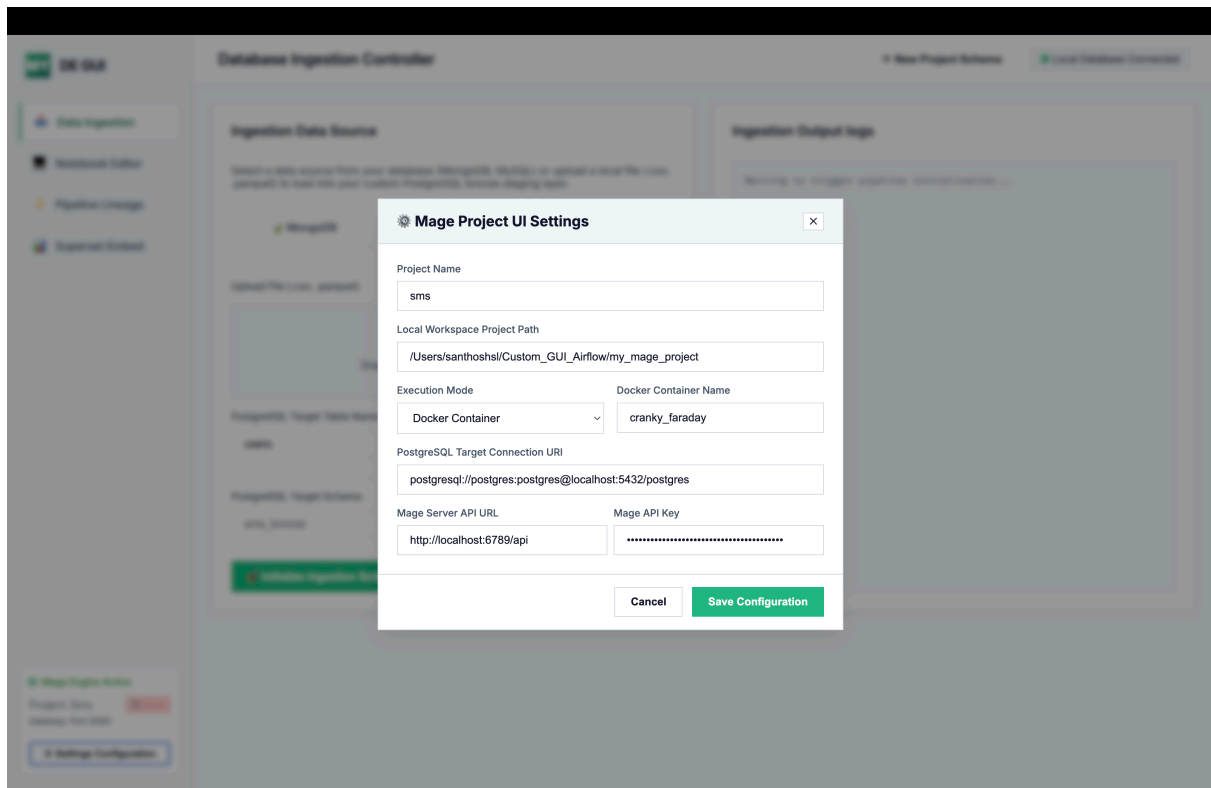


Figure 9 — MySQL settings

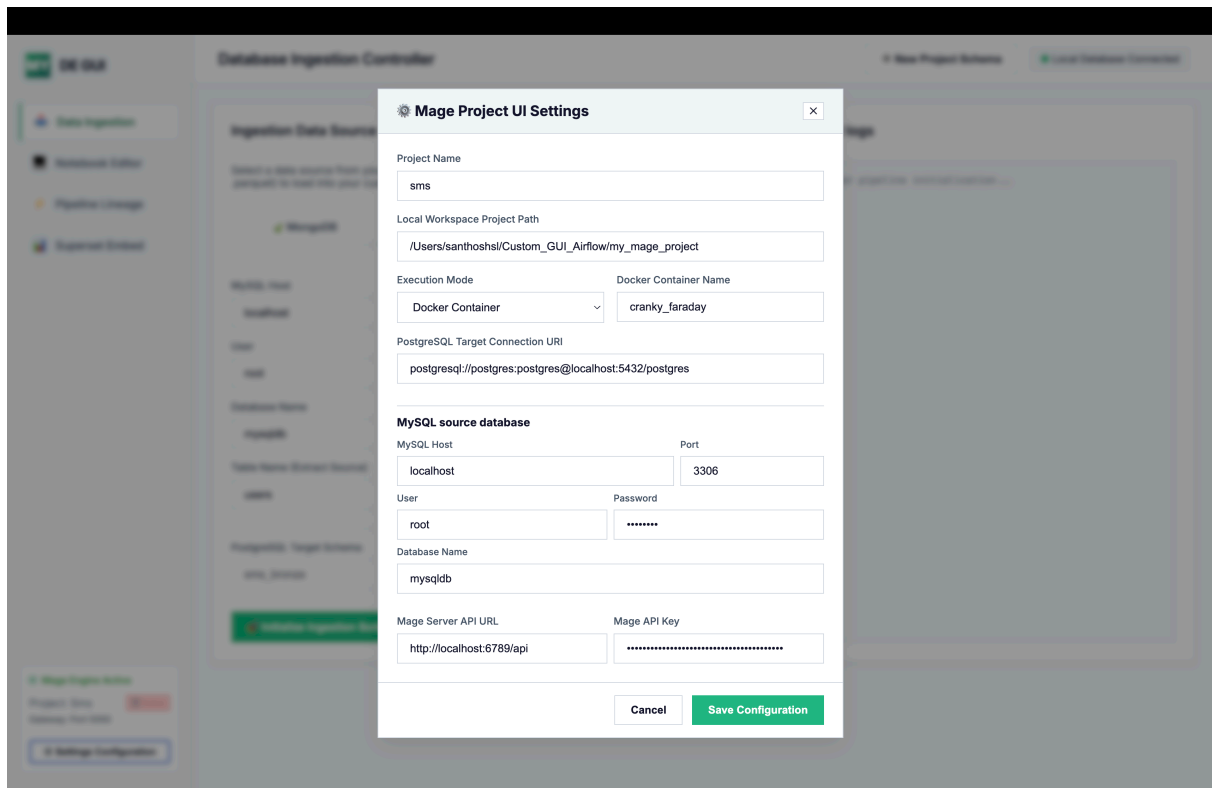
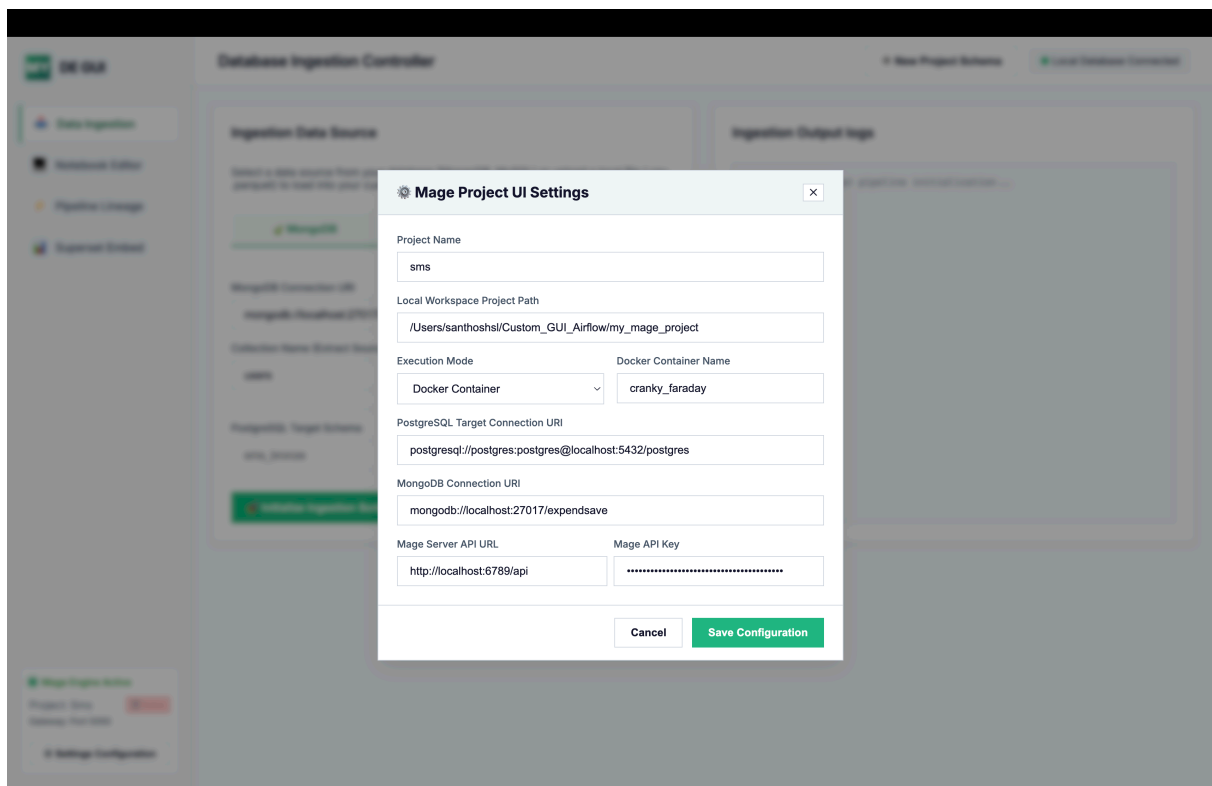


Figure 10 — MongoDB settings



Step 3 — Data ingestion & Bronze script generation

This step builds the raw ingestion layer (`bronze`) by converting source datasets into Postgres tables.

1. Go to `Data Ingestion` .
2. Select the ingestion data source:

A) MongoDB ingestion

- Reads collections from configured Mongo URI.
- Auto-generates an ingestion script under `bronze/` using `templates/mongodb.py` .
- Flattens JSON documents into relational rows and pushes to PostgreSQL.

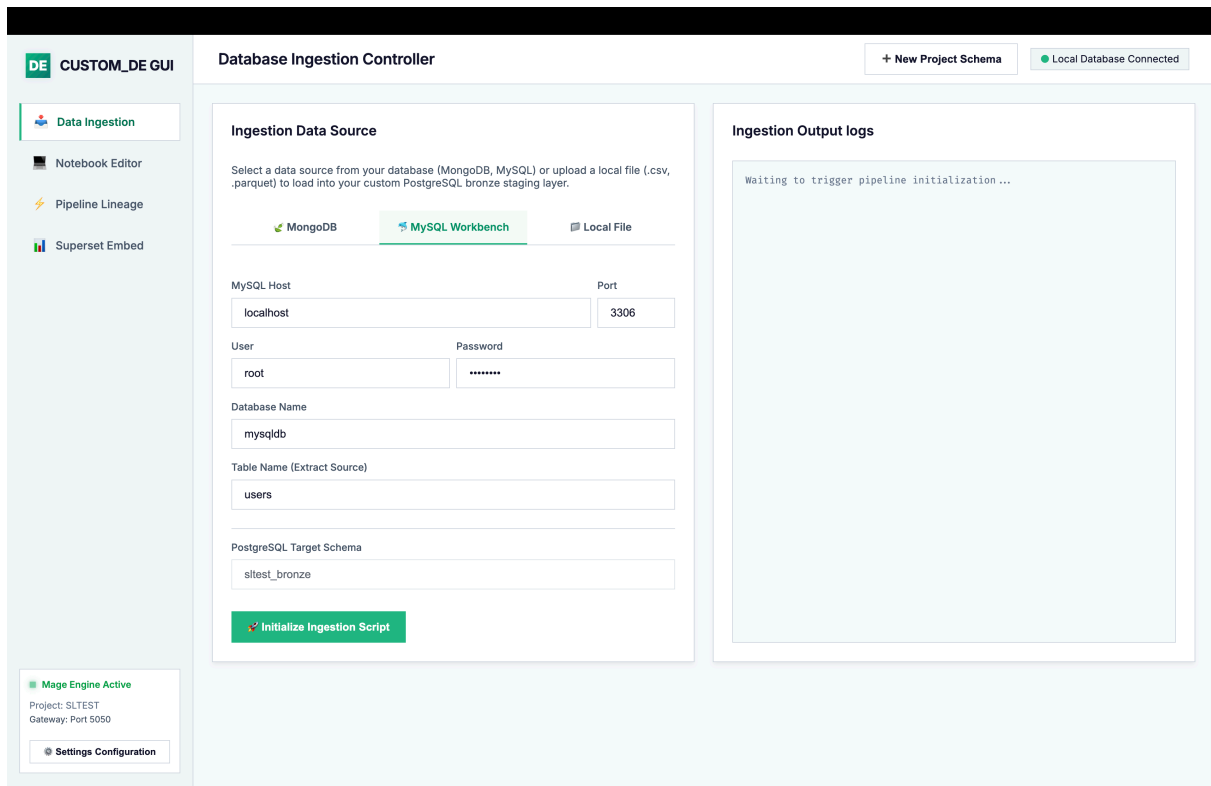
Figure 11 — MongoDB Ingestion

The screenshot displays the 'Database Ingestion Controller' interface. On the left is a sidebar with navigation options: 'Data Ingestion' (selected), 'Notebook Editor', 'Pipeline Lineage', and 'Superset Embed'. Below this is a 'Mage Engine Active' status bar showing 'Project: SLTEST' and 'Gateway: Port 5050', with a 'Settings Configuration' link. The main panel is titled 'Ingestion Data Source' and contains instructions: 'Select a data source from your database (MongoDB, MySQL) or upload a local file (.csv, .parquet) to load into your custom PostgreSQL bronze staging layer.' There are three tabs: 'MongoDB' (active), 'MySQL Workbench', and 'Local File'. Under the 'MongoDB' tab, there are three input fields: 'MongoDB Connection URI' (containing 'mongodb://localhost:27017/expressave'), 'Collection Name (Extract Source)' (containing 'users'), and 'PostgreSQL Target Schema' (containing 'sltest_bronze'). A green 'Initialize Ingestion Script' button is at the bottom of this section. To the right is an 'Ingestion Output logs' panel showing the message 'Waiting to trigger pipeline initialization ...'. At the top right of the main panel are two buttons: '+ New Project Schema' and 'Local Database Connected'.

B) MySQL Workbench ingestion

- Connects using host/port/user/password/database.
- Generates execution script using `templates/mysql.py` (handles host mapping in Docker vs local).

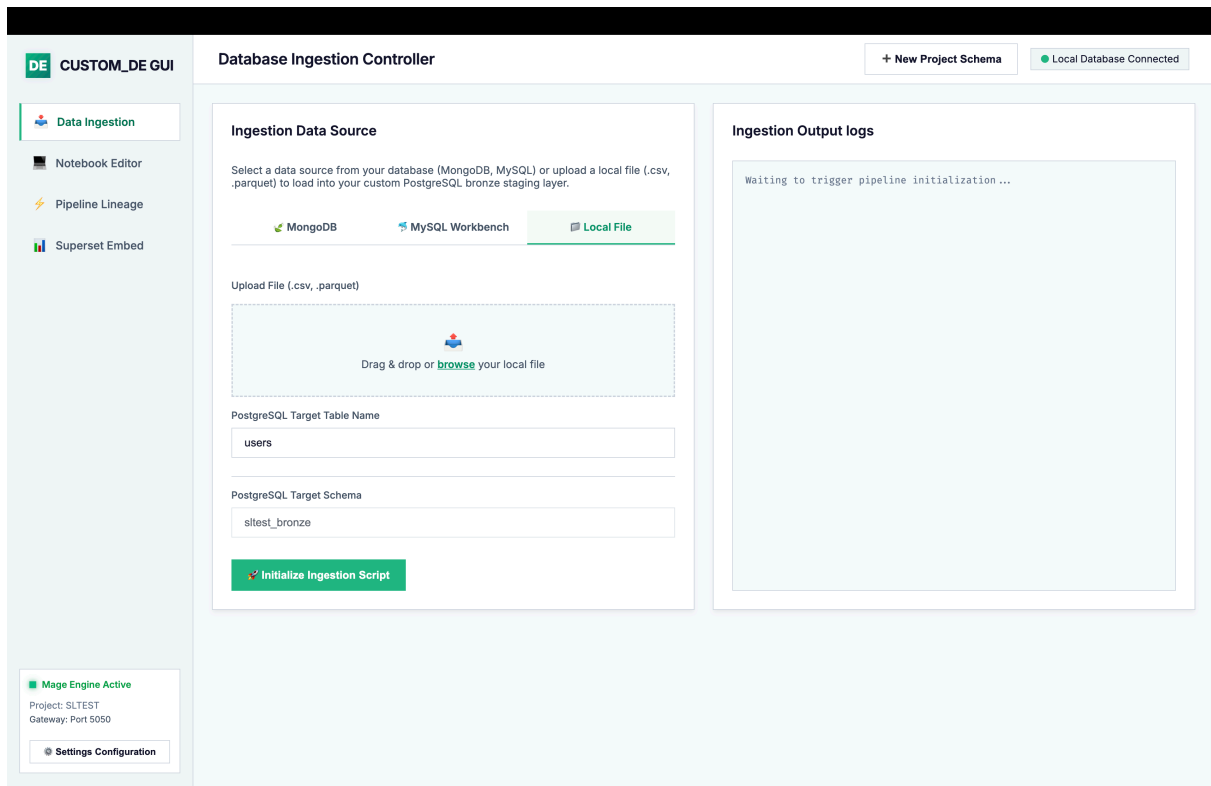
Figure 12 — MYSQL Ingestion



C) Local file ingestion

- Upload `.csv` or `.parquet` and provide a target table name.
- Frontend streams file to `POST /api/ingest/upload` (supports up to **500 MB**).
- File saved under `my_mage_project/{projectName}/bronze/data/`.
- Script auto-generated at `.../bronze/ingest_localfile_{tableName}.py` using `templates/localfile.py`.
- Script loads via `pandas`, normalizes column headers, ensures `{projectName}_bronze` schema, and uploads to Postgres.

Figure 13 — Local file ingestion



3. Click **Confirm & Save Ingestion**.

4. Backend actions

- Saves uploaded file (example): `my_mage_project/sms/bronze/data/spam_data.csv`.
- Injects variables (`POSTGRES_URI`, `FILE_PATH`, table name, etc.) into the chosen template.
- Writes final script to `.../bronze/ingest_[source]_[tablename].py`.
- Registers the new script in the frontend workspace file tree.

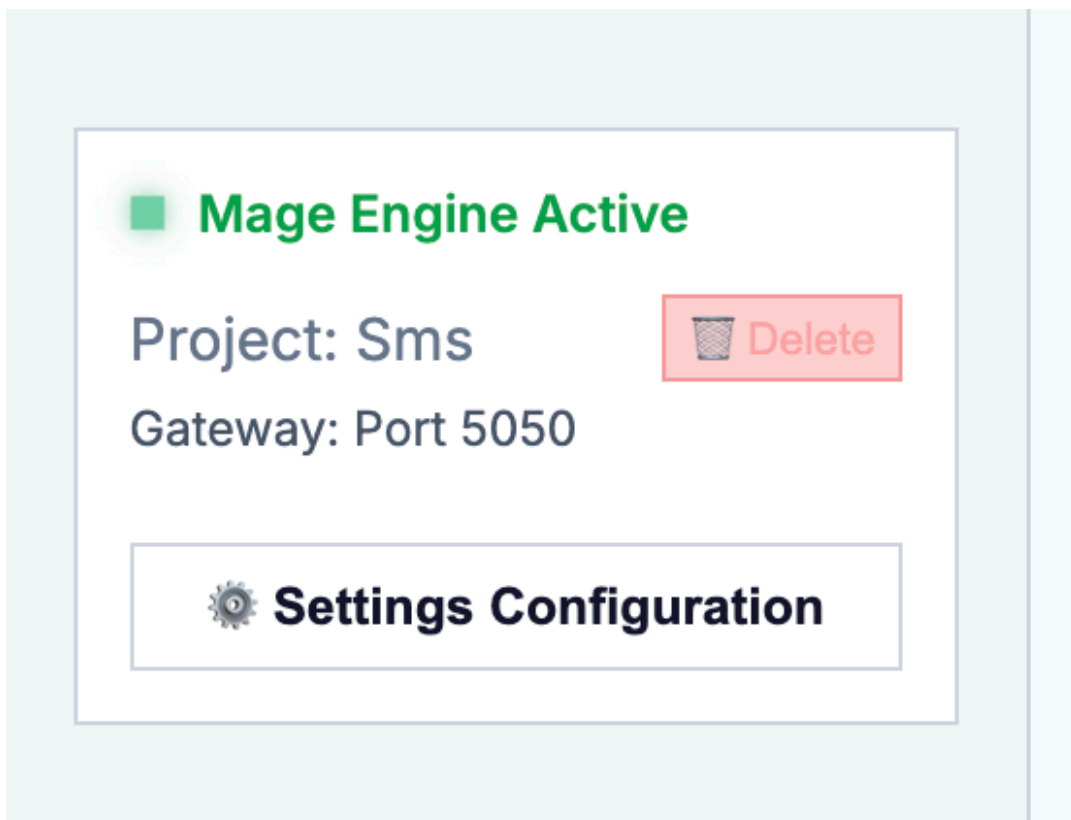
Step 4 — Coding & local script execution (Notebook editor)

1. Navigate to **Notebook Editor**.
2. In the workspace tree, expand `bronze`, `silver`, `gold`.
3. Open a file in the Monaco editor (or create new):
 - `bronze/ingest_...py`
 - `silver/stg_...sql`
 - `gold/fact_...sql`
4. Execute code:

- Click **Run Code** or press **Ctrl + Enter**.
- Verify schemas/tables in pgAdmin4.

 Be careful: using **Delete** in DB settings triggers **DROP DATABASE** and permanently deletes the PostgreSQL database.

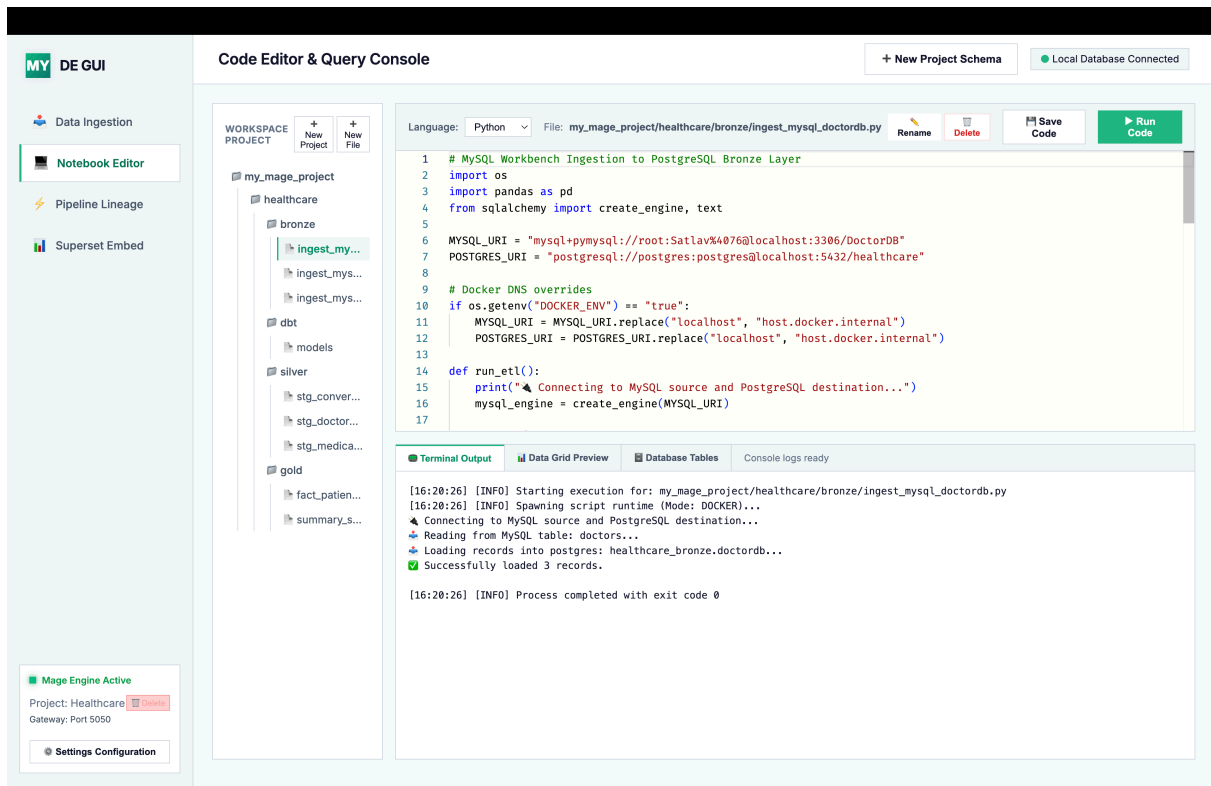
Figure 14 — Run and delete warning UI



Backend actions

- SQL files: spawns background **psql** process and returns tabular JSON into **Data Grid Preview**.
- Python files: spawns docker runtime process, streaming logs to **Terminal Output**.

Figure 15 — Terminal output



Data grid preview

- Click **Database Tables** → select schema/table → **Preview Table**.

Figure 16 — Tables list

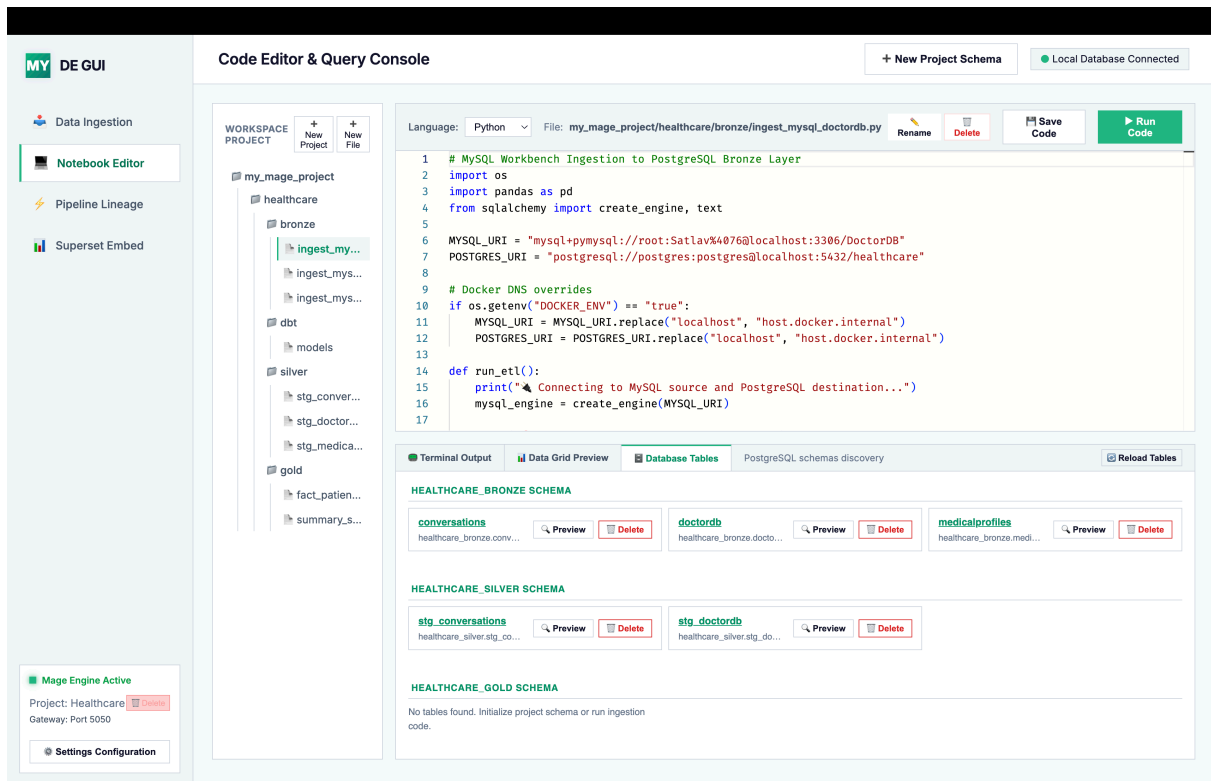
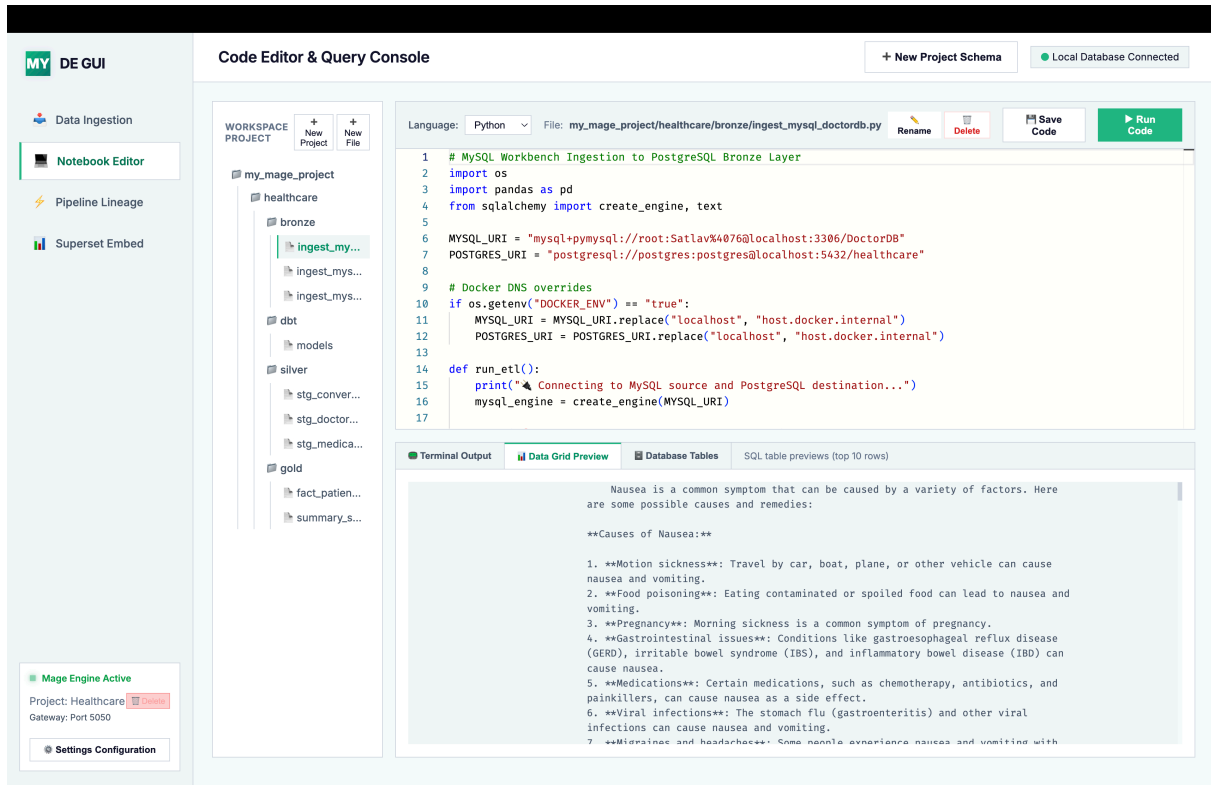


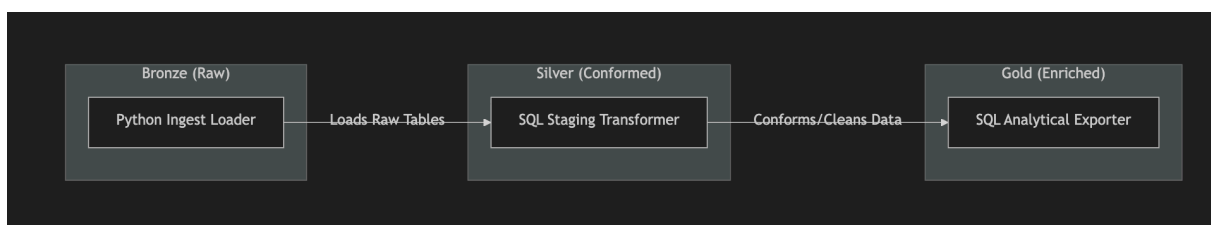
Figure 17 — Table preview



Step 5 — Orchestrating the Medallion run (Pipeline lineage DAG)

Mage OSS compiles a DAG for scheduled execution once blocks are in place.

Figure 18 — Pipeline lineage DAG

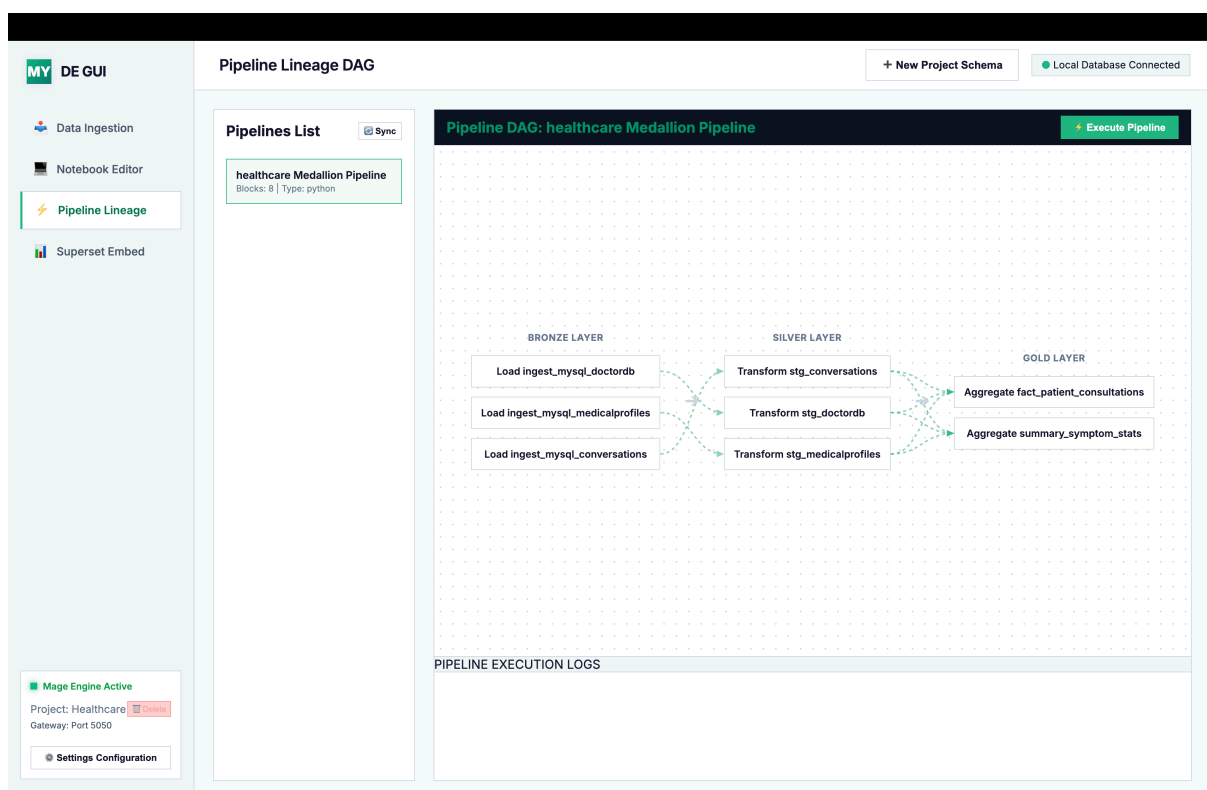


1. Go to **Pipeline Lineage**.
2. Select the active pipeline.
3. Canvas displays upstream/downstream connections (e.g., `ingest_localfile_spam_data` → `stg_spam_data`).
4. Click **Run Pipeline**.

Backend actions

- Gateway proxies to Mage: `/api/pipelines/[uuid]/runs`.
- Execution:
 1. Runs `bronze/` scripts (extract + stage raw)
 2. Compiles and runs DBT models in `silver/`
 3. Populates business metrics in `gold/`
- Logs streamed back to UI.

Figure 19 — Trigger / redeploy options



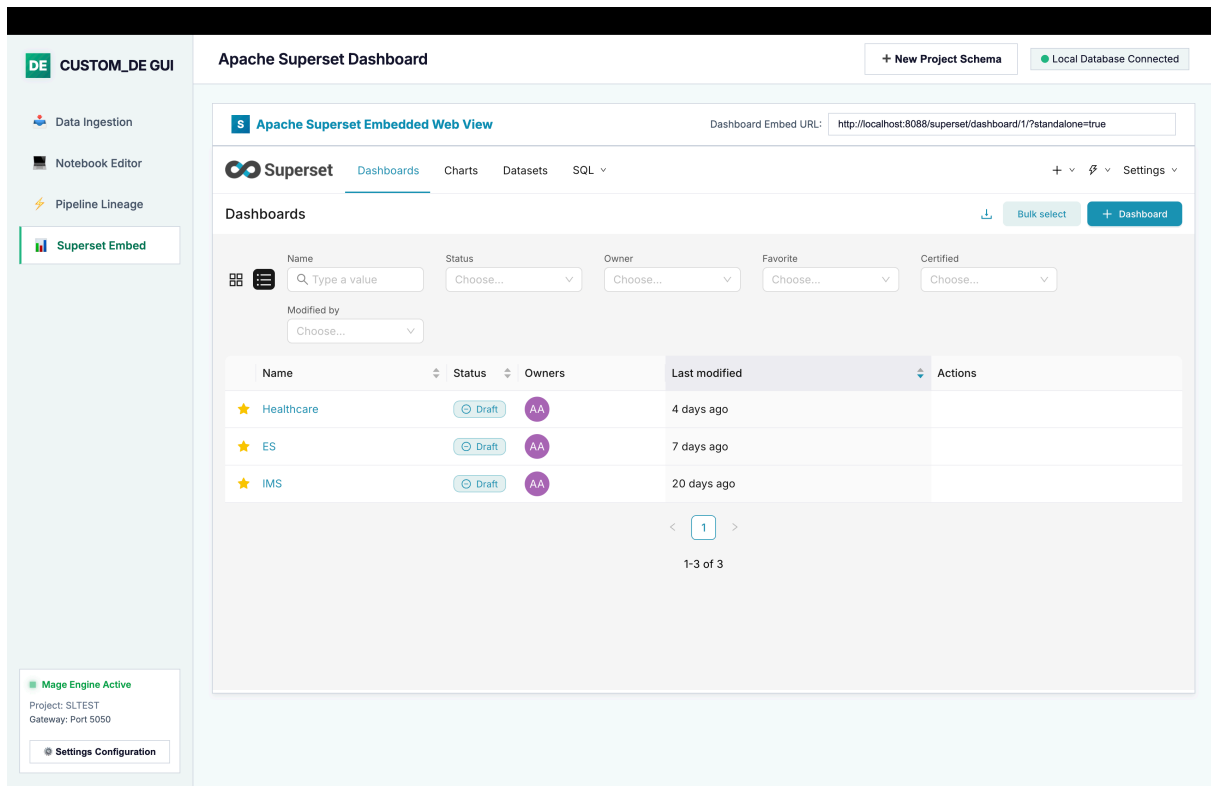
Step 6 — Monitoring & verification (Superset embed)

- Navigate to `Superset Embed` to view live charts and dashboards from Gold tables.

Steps to connect Superset to PostgreSQL

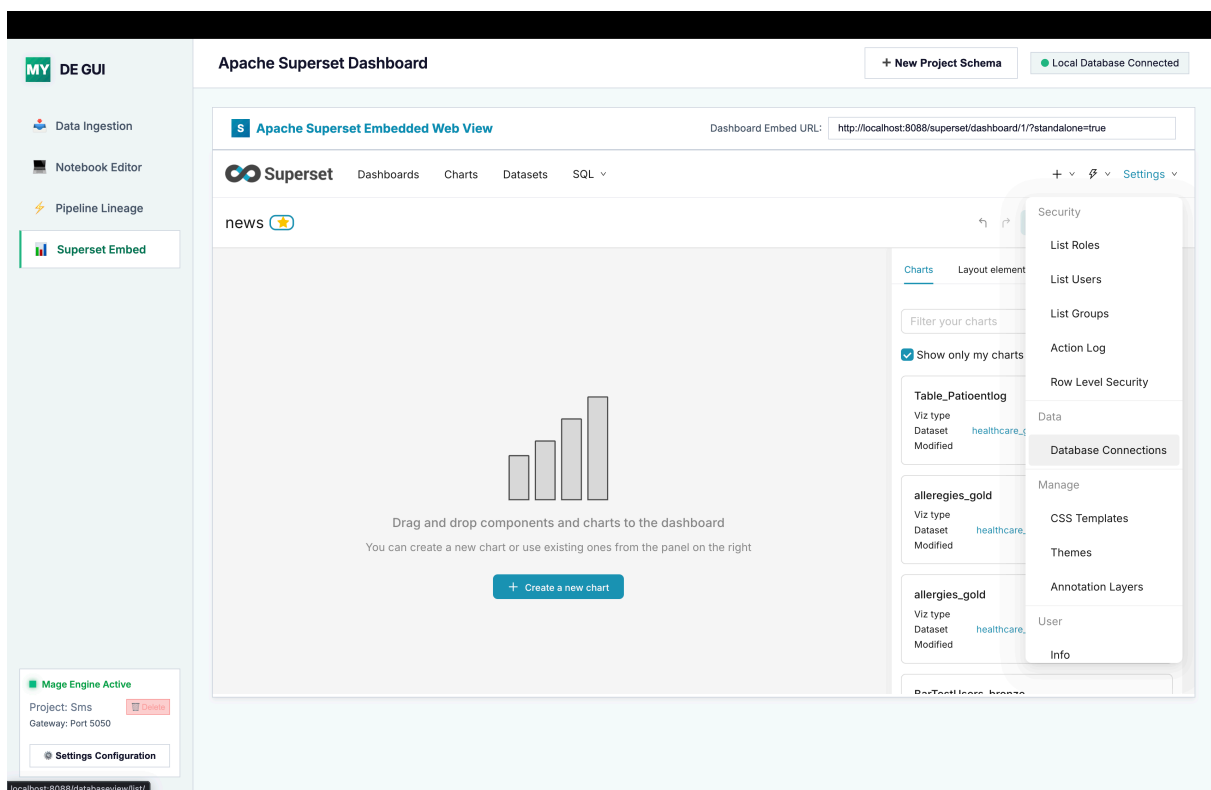
1. Create a new dashboard.

Figure 20 — Create dashboard



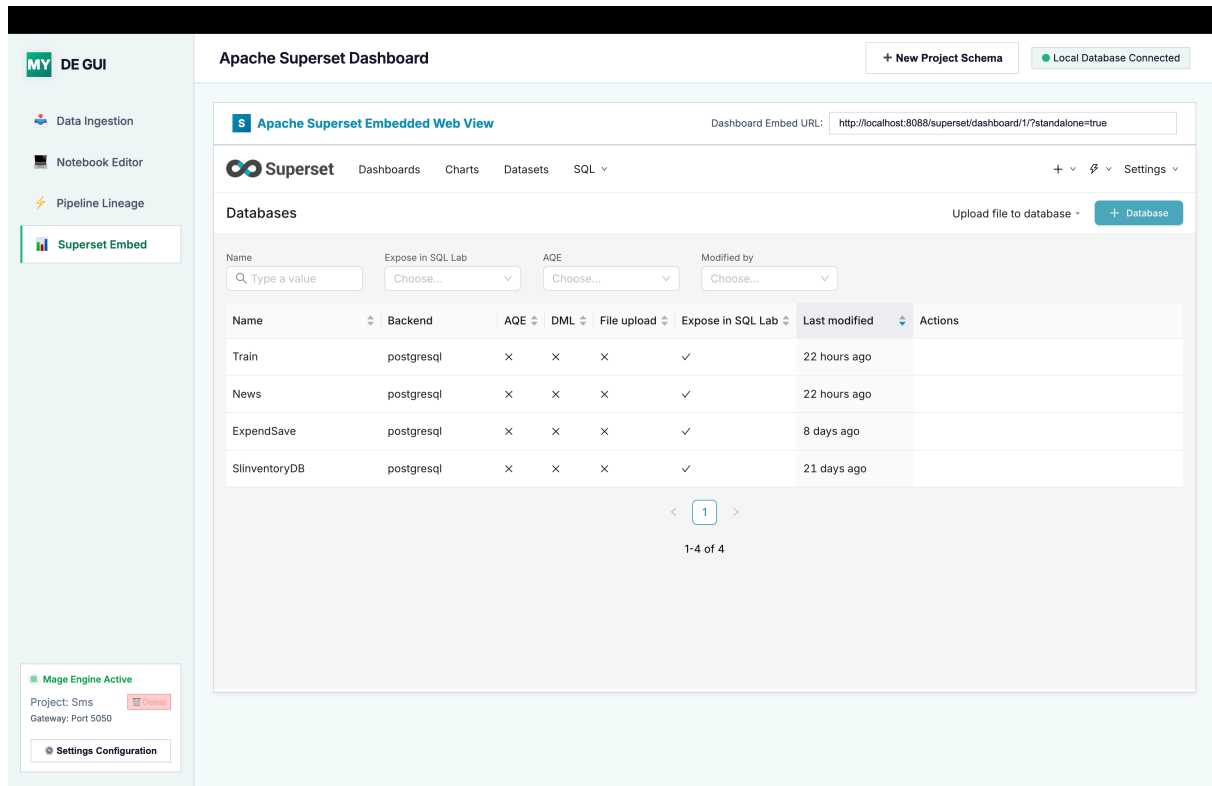
1. Go to **Settings** → **Database connections**.

Figure 21 — Database connections



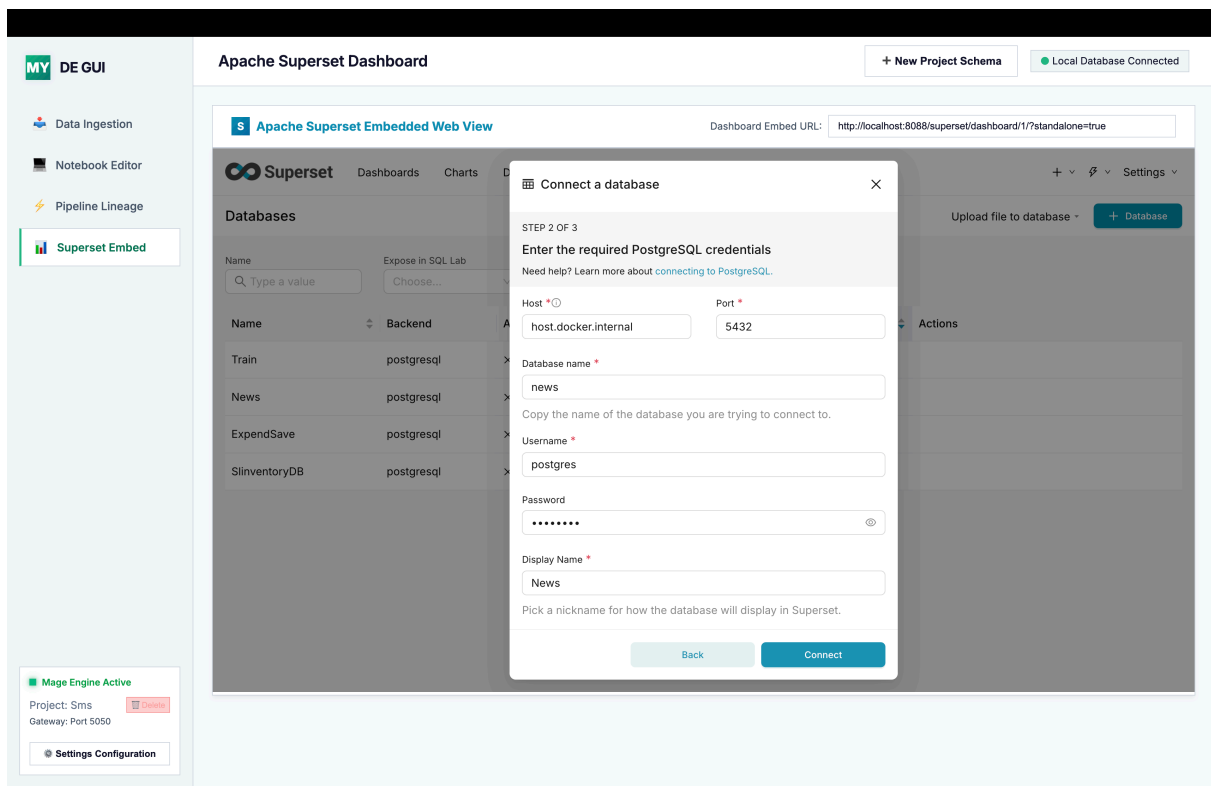
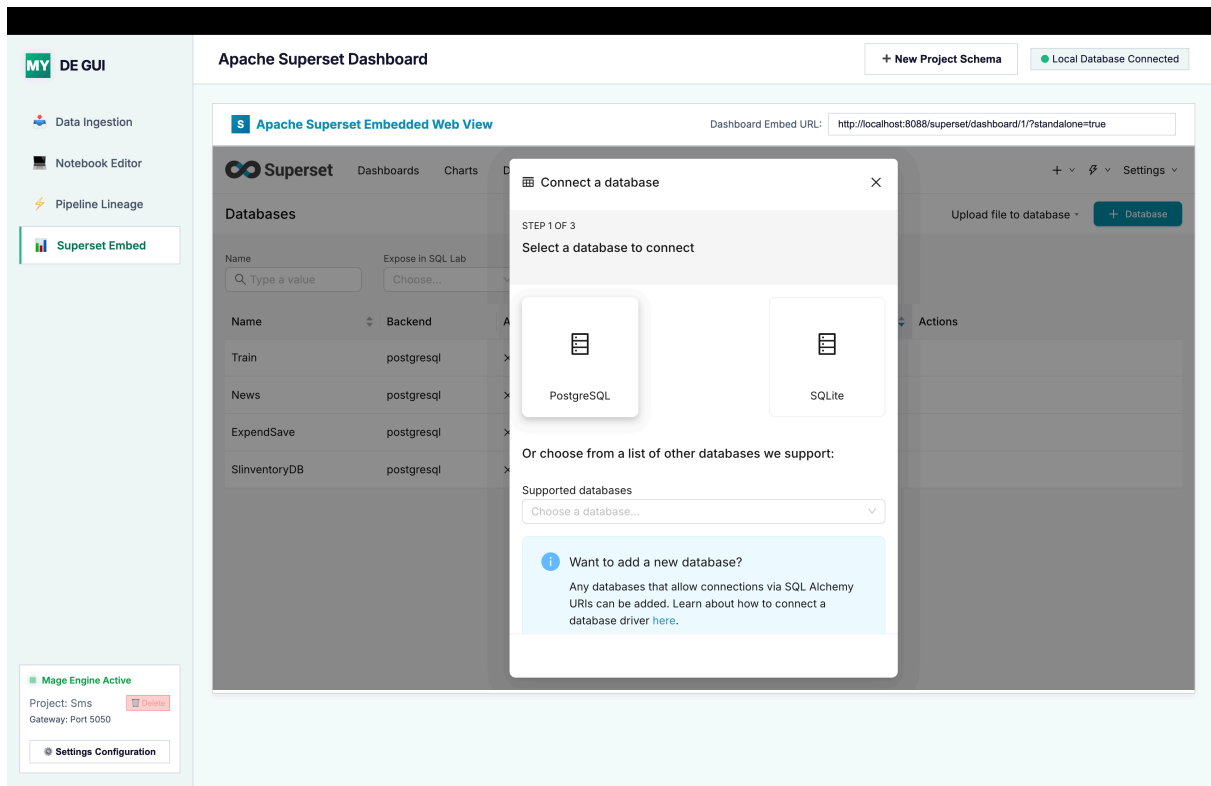
1. Create a new database entry if one isn't selected for the project.

Figure 22 — Select / create database



1. Click **+ Database** → **Postgres**, enter connection details, connect.

Figure 23 — Add Postgres DB



1. Create datasets: **+** → **Data** → **Create dataset** → select schema/table → build charts.

Figure 24 — Create dataset

MY DE GUI

Data Ingestion

Notebook Editor

Pipeline Lineage

Superset Embed

Mage Engine Active

Project: Sms

Gateway: Port 5050

Settings Configuration

Apache Superset Dashboard

+ New Project Schema

Local Database Connected

Apache Superset Embedded Web View

Dashboard Embed URL: http://localhost:8088/superset/dashboard/1/?standalone=true

Superset

Dashboards

Charts

Datasets

SQL

Databases

Q Type a value

Choose...

Choose...

Choose...

Name	Backend	AQE	DML	File upload	Expose in SQL Lab	Last modified
Train	postgresql	x	x	x	✓	22 hours ago
News	postgresql	x	x	x	✓	22 hours ago
ExpendSave	postgresql	x	x	x	✓	8 days ago
SlinventoryDB	postgresql	x	x	x	✓	21 days ago

1

1-4 of 4

Create dataset

Connect Google Sheet

Upload CSV to database

Upload Excel to database

Upload Columnar file to database

Data

SQL query

Chart

Dashboard

MY DE GUI

Data Ingestion

Notebook Editor

Pipeline Lineage

Superset Embed

Mage Engine Active

Project: Sms

Gateway: Port 5050

Settings Configuration

Apache Superset Dashboard

+ New Project Schema

Local Database Connected

Apache Superset Embedded Web View

Dashboard Embed URL: http://localhost:8088/superset/dashboard/1/?standalone=true

Superset

Dashboards

Charts

Datasets

SQL

New dataset

Database

postgresql slinventoryDB

postgresql SlinventoryDB ✓

postgresql ExpendSave

postgresql News

postgresql Train

Select dataset source

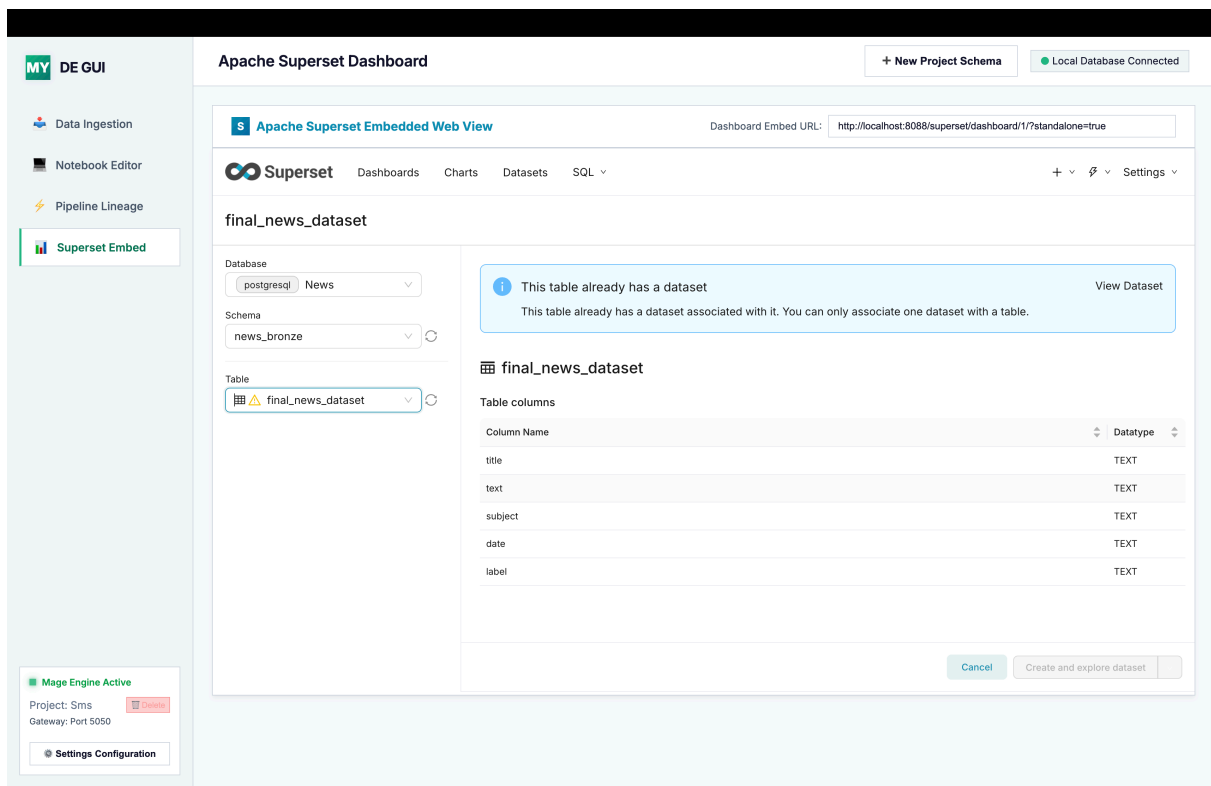
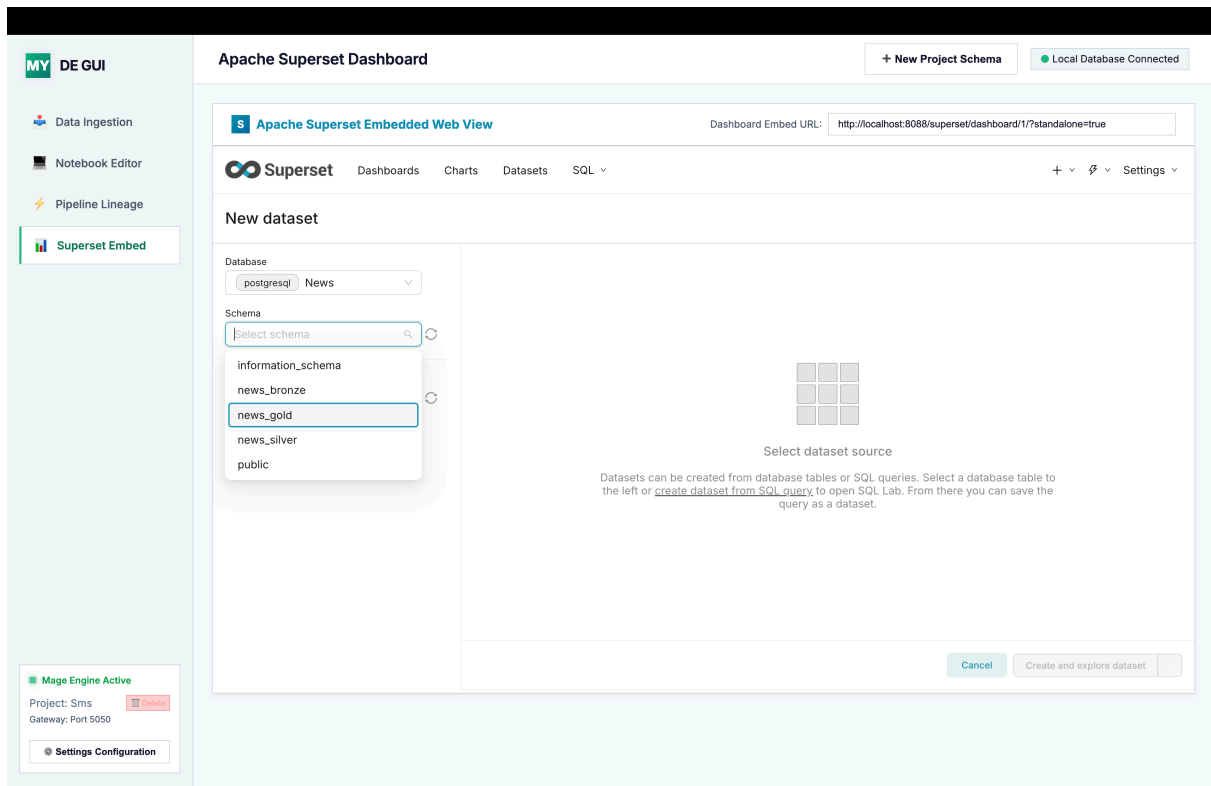
Datasets can be created from database tables or SQL queries. Select a database table to the left or [create dataset from SQL query](#) to open SQL Lab. From there you can save the query as a dataset.

Cancel

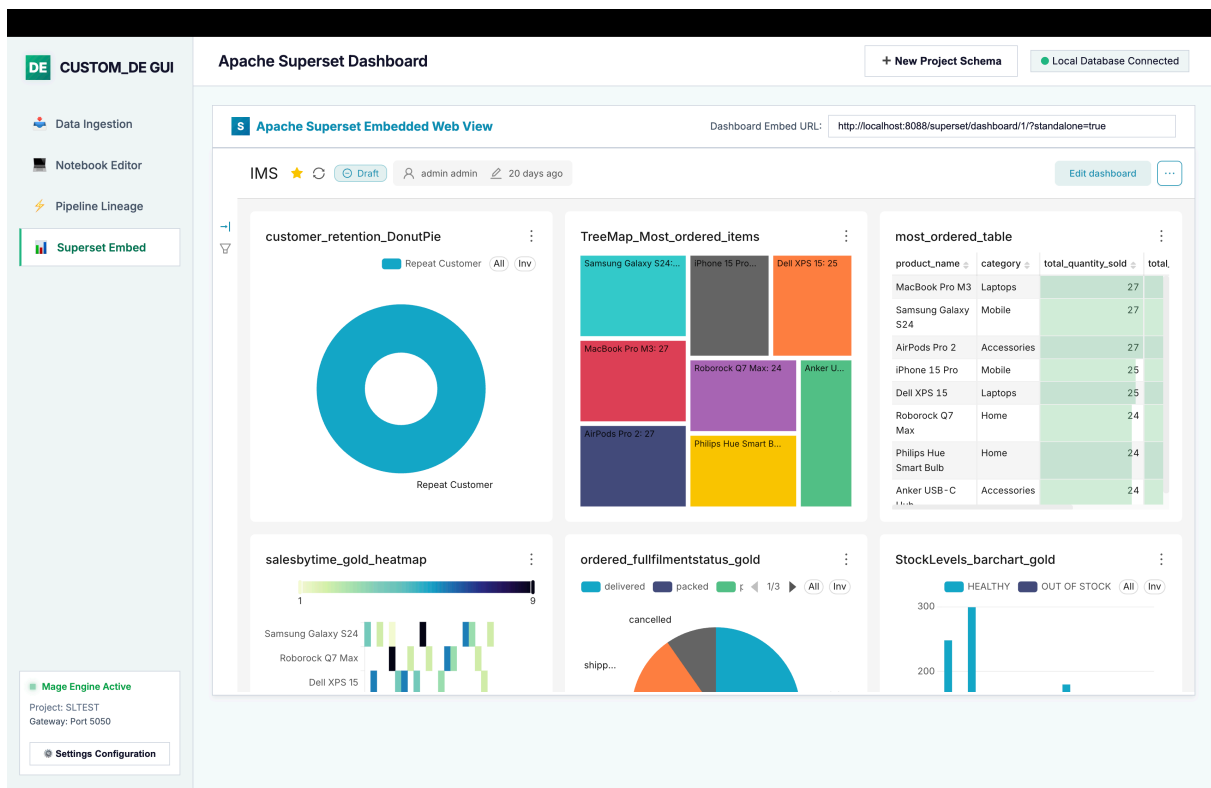
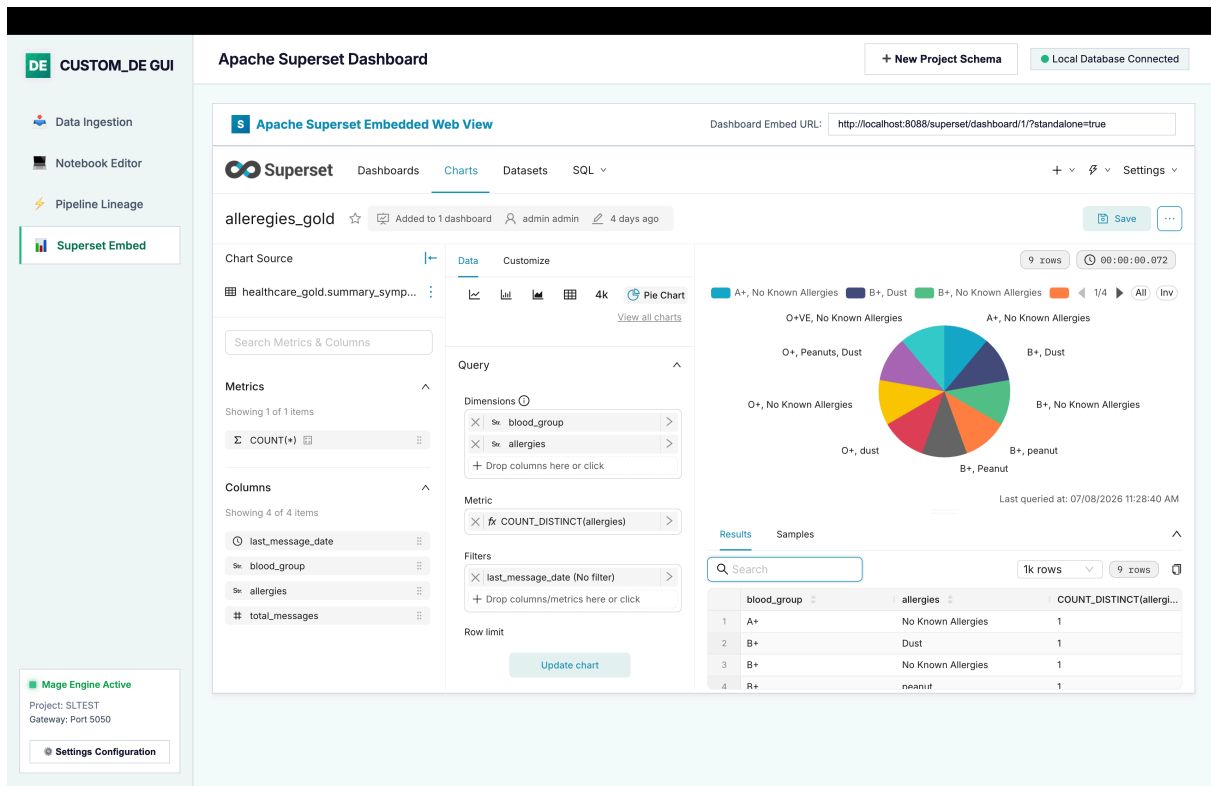
Create and explore dataset

Custom GUI for DE

20



Example dashboard metrics



5) References

1. GitHub (main branch): https://github.com/Santo29SL/CustomDE_UI_POC
(includes Notion user guide link under "About").
2. Apache License 2.0: <https://www.apache.org/licenses/LICENSE-2.0>
3. Mage AI homepage: <https://www.mage.ai/>
4. Mage OSS: <https://www.mage.ai/oss>
5. Mage docs overview: <https://docs.mage.ai/introduction/overview>